

EDUTECH SOLUTION

Artificial Intelligence & Machine Learning Internship

Task 5

Train-Test Split and Evaluation Metrics

using Logistic Regression on Heart Disease Dataset

Submitted by:

MODI DHRUV BHAVESHBHAI

Date of Submission: June 2026

Abstract

This report presents a detailed account of Task 5 of the Edutech Solution Artificial Intelligence and Machine Learning Internship, which focuses on the implementation of the train-test split methodology and the application of standard classification evaluation metrics on the Heart Disease Dataset. The objective of this task is to build foundational competency in machine learning model evaluation — a skill that is central to the development of reliable, production-ready predictive systems.

The Heart Disease Dataset, sourced from Kaggle, contains 918 patient records described by 11 clinical features including age, sex, chest pain type, resting blood pressure, cholesterol levels, fasting blood sugar, resting electrocardiogram results, maximum heart rate, exercise-induced angina, ST depression, and slope of the peak exercise ST segment. The target variable is a binary indicator of heart disease presence (1) or absence (0).

After loading the dataset, categorical features were encoded using the pandas `get_dummies()` function to convert them into a numerical format suitable for logistic regression. The data was then divided into training (80%) and testing (20%) subsets using Scikit-learn's `train_test_split` utility, yielding 734 training records and 184 testing records. A Logistic Regression classifier was trained on the scaled training data and subsequently evaluated on the held-out test set.

The model achieved an accuracy of 84.78%, a precision of 90.72%, and a recall of 82.24% for the positive class. The confusion matrix revealed 88 true positives, 68 true negatives, 9 false positives, and 19 false negatives. These results demonstrate that the logistic regression model provides a reliable baseline for heart disease classification and that the evaluation framework employed offers meaningful insight into the model's clinical usefulness.

Table of Contents

Declaration.....	Error! Bookmark not defined.
Acknowledgement	Error! Bookmark not defined.
Abstract.....	2
Table of Contents.....	3
Chapter 1: Introduction	6
1.1 Artificial Intelligence	6
1.2 Machine Learning	6
1.3 Supervised Learning	6
1.4 Classification Problems	7
1.5 Logistic Regression.....	7
1.6 Importance of Model Evaluation	7
1.7 Objectives of Task 5	8
Chapter 2: Dataset Description	9
2.1 Dataset Overview.....	9
2.2 Dataset Source	9
2.3 Dataset Characteristics.....	9
2.4 Dataset Shape.....	9
2.5 Feature Description.....	10
2.6 Target Variable	10
2.7 Importance of Heart Disease Prediction	11
2.8 Dataset Understanding	11
Chapter 3: Methodology	12
3.1 Dataset Loading	12
3.2 Data Inspection	12
3.3 Feature and Target Selection	12
3.4 Data Encoding.....	12
3.5 Train-Test Split	13
3.6 Logistic Regression.....	13
3.6.1 Working Principle.....	13
3.6.2 Mathematical Concept	13
3.6.3 Advantages.....	14
3.6.4 Limitations	14
3.7 Prediction Process.....	14
Chapter 4: Implementation	15
4.1 Development Environment	15

4.1.1 Google Colab	15
4.1.2 Python	15
4.1.3 Pandas	15
4.1.4 Scikit-learn	15
4.2 GitHub Repository Usage	16
4.3 Code Explanation	16
Block 1: Importing Libraries	16
Block 2: Loading and Encoding the Dataset	16
Block 3: Feature and Target Selection with Train-Test Split	17
Block 4: Scaling, Training, and Prediction	17
Block 5: Evaluation Metrics	17
4.4 Execution Workflow	18
Chapter 5: Results and Evaluation	19
5.1 Model Output Summary	19
5.2 Accuracy Score	19
5.3 Precision Score	19
5.4 Recall Score	19
5.5 Confusion Matrix	19
5.6 Model Performance Interpretation	21
Chapter 6: Discussion	22
6.1 Importance of Train-Test Split	22
6.2 Importance of Accuracy	22
6.3 Importance of Precision	22
6.4 Importance of Recall	22
6.5 Importance of Confusion Matrix	23
6.6 Advantages of Logistic Regression	23
6.7 Challenges Faced During Implementation	23
6.8 Lessons Learned	23
6.9 Future Improvements	24
Chapter 7: Interview Questions and Answers	25
Question 1: What is Overfitting?	25
Question 2: Accuracy vs Precision?	25
Question 3: What is Recall?	26
Question 4: What is a Confusion Matrix?	26
Question 5: Why Split Data?	26
Chapter 8: Conclusion	28
8.1 Summary of Work	28

8.2 Key Findings	28
8.3 Skills Learned	28
8.4 Importance of Model Evaluation	28
8.5 Final Conclusion	29
References	30
Appendix A: Complete Python Source Code	31
Appendix B: Dataset Feature Description Table	33

Chapter 1: Introduction

1.1 Artificial Intelligence

Artificial Intelligence (AI) is a broad field of computer science concerned with the creation of systems capable of performing tasks that typically require human cognitive abilities. These tasks include recognising speech, understanding natural language, making decisions, perceiving visual environments, and translating between languages. AI encompasses a spectrum of approaches ranging from rule-based expert systems, which rely on explicitly programmed logic, to modern data-driven approaches that allow machines to learn from experience. In recent decades, AI has moved from academic laboratories into everyday applications, powering recommendation engines, medical diagnostic tools, autonomous vehicles, and language models, fundamentally reshaping how humans interact with technology.

1.2 Machine Learning

Machine Learning (ML) is a subfield of artificial intelligence that enables systems to learn and improve from data without being explicitly programmed for each task. Rather than following hand-crafted rules, a machine learning algorithm identifies statistical patterns and relationships in training data and uses these patterns to make predictions or decisions on new, unseen data. Machine learning is broadly categorised into three paradigms: supervised learning, where the model learns from labelled examples; unsupervised learning, where the model discovers hidden structure in unlabelled data; and reinforcement learning, where an agent learns by interacting with an environment and receiving feedback in the form of rewards or penalties.

1.3 Supervised Learning

Supervised learning is the most widely used machine learning paradigm in practical applications. In supervised learning, the training dataset consists of input-output pairs (also called feature-label pairs), and the model's objective is to learn a mapping function from inputs to outputs. Once trained, the model can predict the output for previously unseen inputs. Supervised learning tasks are further divided into two categories: regression, where the output is a continuous numerical value, and classification, where the output belongs to one of a finite set of discrete categories. Heart disease prediction, salary estimation, spam detection, and image recognition are all examples of supervised learning applications.

1.4 Classification Problems

Classification is a supervised learning task in which the goal is to assign an input to one of a predefined set of categories. When there are only two possible categories (such as disease present versus absent, or spam versus not spam), the problem is called binary classification. When there are three or more categories, it is called multi-class classification. Classification algorithms learn a decision boundary in the feature space that separates different classes. Common classification algorithms include Logistic Regression, Decision Trees, Random Forests, Support Vector Machines, k-Nearest Neighbours, and Neural Networks. In this task, binary classification is applied to predict whether a patient has heart disease based on clinical measurements.

1.5 Logistic Regression

Logistic Regression is a statistical classification algorithm that estimates the probability that a given input belongs to a particular class. Despite its name, logistic regression is a classification model, not a regression model. It uses the sigmoid (logistic) activation function to constrain the output of a linear equation to the range $[0, 1]$, which can be interpreted as the probability of belonging to the positive class. When the predicted probability exceeds a decision threshold (typically 0.5), the sample is classified as positive; otherwise, it is classified as negative. Logistic regression is widely used as a baseline model in binary classification tasks due to its simplicity, interpretability, and computational efficiency.

1.6 Importance of Model Evaluation

Training a machine learning model is only one part of the model development lifecycle. Of equal importance is the rigorous evaluation of the trained model to determine whether it generalises well to unseen data and whether its performance meets the requirements of the target application. Model evaluation involves splitting data into independent training and testing partitions, computing performance metrics such as accuracy, precision, and recall, and interpreting these metrics in the context of the problem domain. In medical applications such as heart disease prediction, the cost of different types of classification errors is asymmetric, making metrics such as recall and the confusion matrix particularly important for clinical decision-making.

1.7 Objectives of Task 5

The objectives of Task 5 of the Edutech Solution internship are as follows. First, to load and inspect the Heart Disease Dataset and understand its structure, feature types, and class distribution. Second, to perform necessary data preprocessing including categorical feature encoding. Third, to split the dataset into training and testing sets and to explain the purpose and benefits of this methodology. Fourth, to train a Logistic Regression classifier on the training data. Fifth, to generate predictions on the test data. Sixth, to compute and interpret the accuracy, precision, recall, and confusion matrix. Seventh, to demonstrate a comprehensive understanding of model evaluation fundamentals through professional documentation.

Chapter 2: Dataset Description

2.1 Dataset Overview

The Heart Disease Dataset used in this task is a well-known benchmark dataset in the field of medical machine learning. It aggregates patient records from four major clinical centres and has been standardised into a single consistent format for research and educational use. The dataset is suitable for binary classification tasks and provides a realistic representation of the clinical features associated with cardiovascular disease.

2.2 Dataset Source

The dataset was obtained from Kaggle at the following URL: <https://www.kaggle.com/code/tanmay111999/heart-failure-prediction-cv-score-90-5-models/input>. The original data has been compiled from contributions by the Cleveland Clinic Foundation, the Hungarian Institute of Cardiology, the University Hospital in Zurich, and the Long Beach VA Medical Center. The GitHub repository for this project is available at <https://github.com/dbmodi2311/Data-Collection-Data-Source-Identification/tree/main/Task%205>, and the Google Colab notebook implementation is hosted at https://colab.research.google.com/drive/1Dzrh7EupG_FMEAyvNVKhzGMj7YNIBifb.

2.3 Dataset Characteristics

The dataset is a structured, tabular dataset with a mix of numerical and categorical variables. All records are complete and no missing values are present in the version used in this task, which eliminates the need for imputation preprocessing. The binary nature of the target variable makes the dataset directly applicable to logistic regression and other binary classification algorithms.

2.4 Dataset Shape

The dataset contains 918 rows (patient records) and 12 columns. Of the 12 columns, 11 represent independent features used to describe each patient's clinical profile, and one column — HeartDisease — serves as the dependent target variable. This results in 918 labelled training examples available for model development.

2.5 Feature Description

The following table provides a complete description of all 12 features in the dataset, including their data type, possible values, and clinical interpretation.

Table 1: Heart Disease Dataset Feature Descriptions

Feature	Type	Values / Range	Clinical Description
Age	Numerical	28 – 77 years	Age of the patient in years
Sex	Categorical	M, F	Biological sex of the patient
ChestPainType	Categorical	ATA, NAP, ASY, TA	Chest pain type: Atypical Angina, Non-Anginal Pain, Asymptomatic, Typical Angina
RestingBP	Numerical	0 – 200 mm Hg	Resting blood pressure on admission
Cholesterol	Numerical	0 – 603 mg/dl	Serum cholesterol level
FastingBS	Binary	0, 1	Fasting blood sugar > 120 mg/dl (1 = True)
RestingECG	Categorical	Normal, ST, LVH	Resting electrocardiogram result
MaxHR	Numerical	60 – 202 bpm	Maximum heart rate achieved during stress test
ExerciseAngina	Categorical	Y, N	Exercise-induced chest pain
Oldpeak	Numerical	-2.6 – 6.2	ST depression induced by exercise relative to rest
ST_Slope	Categorical	Up, Flat, Down	Slope of peak exercise ST segment
HeartDisease	Binary (Target)	0, 1	1 = heart disease present, 0 = absent

2.6 Target Variable

The target variable, HeartDisease, is a binary column encoding the clinical diagnosis outcome. A value of 1 indicates that the patient has been diagnosed with heart disease, while a value of 0 indicates no diagnosis of heart disease. In the dataset, 508 records carry the positive label (55.3%) and 410 records carry the negative label (44.7%). This moderate class imbalance is representative of real-world clinical datasets and slightly favours the positive class, which is an important consideration when interpreting precision and recall values.

2.7 Importance of Heart Disease Prediction

Cardiovascular disease is the leading cause of mortality globally, responsible for approximately 17.9 million deaths per year according to the World Health Organization. Early identification of individuals at risk of heart disease can significantly improve clinical outcomes by enabling timely interventional strategies such as lifestyle modification, pharmacotherapy, or surgical intervention. Machine learning models trained on clinical data can serve as decision-support tools for healthcare professionals, flagging high-risk patients for further evaluation. The Heart Disease Dataset provides a compact yet clinically meaningful set of easily measurable features, making it an excellent testbed for evaluating classification algorithms.

2.8 Dataset Understanding

An analysis of the dataset reveals several noteworthy patterns. Patients with asymptomatic chest pain (ASY) have a substantially higher prevalence of heart disease than those with typical or atypical angina. Older patients and male patients are more frequently represented in the positive class. A flat ST slope and high Oldpeak values are associated with elevated risk. These patterns align with well-established clinical knowledge, which provides a degree of face validity to the dataset and supports the use of logistic regression as an interpretable classifier whose coefficients can be related back to clinical risk factors.

Chapter 3: Methodology

3.1 Dataset Loading

The first step in the implementation was to load the Heart Disease Dataset from a comma-separated values (CSV) file into a Pandas DataFrame using the `pd.read_csv()` function. The Pandas library provides a highly efficient and flexible data structure for tabular data manipulation, and the DataFrame format makes it straightforward to inspect, filter, transform, and split the data. After loading, the DataFrame was inspected using `df.shape` to confirm the expected dimensions of 918 rows and 12 columns.

3.2 Data Inspection

Following loading, basic exploratory data analysis was conducted to understand the structure and quality of the data. The `df.head()` method was used to preview the first few records, `df.info()` was used to examine column data types and check for null values, and `df.describe()` was used to compute summary statistics for numerical features. This step confirmed that the dataset contains no missing values and that the features span a range of scales, which motivated the use of feature scaling prior to logistic regression training.

3.3 Feature and Target Selection

The feature matrix X was constructed by dropping the `HeartDisease` column from the DataFrame using `df.drop('HeartDisease', axis=1)`. The target vector y was extracted as `df['HeartDisease']`. This separation of features and labels is a prerequisite for all supervised learning workflows in Scikit-learn, where the model's `fit()` method expects features and labels as separate arguments.

3.4 Data Encoding

The dataset contains several categorical (non-numerical) columns — `Sex`, `ChestPainType`, `RestingECG`, `ExerciseAngina`, and `ST_Slope` — that must be converted into a numerical format before they can be processed by the logistic regression algorithm. In this implementation, the `pd.get_dummies()` function from the Pandas library was used to perform one-hot encoding. One-hot encoding creates a new binary (0 or 1) column for each unique category within a categorical variable. For example, the `ChestPainType` column with four categories (`ATA`, `NAP`, `ASY`, `TA`) is transformed into four separate binary columns: `ChestPainType_ATA`, `ChestPainType_NAP`, `ChestPainType_ASY`, and `ChestPainType_TA`.

One-hot encoding is preferred over label encoding for nominal categorical variables (variables with no natural ordering) because it avoids introducing false ordinal relationships. The `drop_first=True` parameter may be used to eliminate one category per feature and prevent multicollinearity (also known as the dummy variable trap). After encoding, the DataFrame is composed entirely of numerical columns ready for model training.

3.5 Train-Test Split

The encoded dataset was split into training and testing subsets using Scikit-learn's `train_test_split()` function with a `test_size` parameter of 0.2 (20%) and a `random_state` of 42 for reproducibility. This resulted in 734 samples in the training set and 184 samples in the test set.

The purpose of the train-test split is to simulate a deployment scenario in which the model is evaluated on data it has never encountered during training. Training and evaluating a model on the same data would produce an optimistically biased performance estimate, as the model would effectively have memorised the training patterns. The 80:20 ratio is a widely adopted convention in machine learning that balances the competing needs of having sufficient data for training (to avoid underfitting) and a representative, statistically meaningful test set (to produce reliable performance estimates). With 918 records, a 20% test set yields 184 samples — a size that provides stable estimates of accuracy, precision, and recall while leaving 734 samples for training.

3.6 Logistic Regression

3.6.1 Working Principle

Logistic Regression models the relationship between the input features and the binary target variable by estimating the probability that an instance belongs to the positive class. It applies a linear transformation to the feature vector and then passes the result through the sigmoid function, which maps any real value to the (0, 1) interval.

3.6.2 Mathematical Concept

The sigmoid function is defined as: $\sigma(z) = 1 / (1 + e^{(-z)})$, where $z = w_0 + w_1 * x_1 + w_2 * x_2 + \dots + w_n * x_n$ is the linear combination of features and learned weights. The model's parameters (weights w) are optimised by minimising the binary cross-entropy (log-loss) function using gradient-based optimisation algorithms. The decision rule classifies a sample as positive if $\sigma(z) \geq 0.5$ and as negative otherwise.

3.6.3 Advantages

Logistic Regression offers several advantages for this task. It is computationally efficient and can be trained rapidly even on moderately large datasets. Its coefficients are directly interpretable as log-odds ratios, enabling clinical insight into which features most strongly influence the prediction. It naturally produces calibrated probability outputs rather than just hard class labels. It supports regularisation (L1 and L2) to prevent overfitting. It serves as a strong, well-understood baseline against which more complex models can be compared.

3.6.4 Limitations

The primary limitations of logistic regression are its assumption of a linear decision boundary, which may be insufficient for datasets with complex non-linear relationships between features and the target. It can also be sensitive to outliers in the feature space. In datasets with highly correlated features (multicollinearity), the coefficient estimates may become unstable. Despite these limitations, logistic regression is widely used as a first-choice model in clinical predictive analytics due to its transparency and regulatory acceptability.

3.7 Prediction Process

After training, the fitted model was used to generate class predictions for each record in the test set using `model.predict(X_test)`. This method applies the learned decision boundary to each test sample and returns a predicted class label of 0 or 1. The predicted labels were then compared against the true test labels (`y_test`) to compute all evaluation metrics. The complete set of predictions was also passed to Scikit-learn's `confusion_matrix()` and `classification_report()` functions to generate a comprehensive evaluation summary.

Chapter 4: Implementation

4.1 Development Environment

4.1.1 Google Colab

The implementation was carried out entirely in Google Colaboratory (Colab), a cloud-based Jupyter Notebook environment provided by Google. Colab offers free access to Python runtime environments with pre-installed data science libraries, making it an ideal platform for machine learning prototyping without local environment configuration. It supports GPU and TPU acceleration for deep learning workloads and allows easy sharing of notebooks via Google Drive. For this task, Colab provided a reproducible, browser-accessible environment for running the complete machine learning pipeline.

4.1.2 Python

Python 3 was the programming language used throughout this project. Python has become the dominant language for machine learning and data science due to its clean syntax, extensive ecosystem of scientific computing libraries, and the availability of frameworks such as Scikit-learn, TensorFlow, and PyTorch. Python's interactive notebook compatibility with Jupyter and Colab makes it particularly well-suited for exploratory data analysis and iterative model development.

4.1.3 Pandas

The Pandas library was used for all data loading, inspection, and preprocessing operations. Its DataFrame data structure provides a two-dimensional labelled table that supports efficient indexing, filtering, aggregation, and transformation operations. The `pd.read_csv()` function was used to load the dataset, and `pd.get_dummies()` was used for one-hot encoding of categorical features.

4.1.4 Scikit-learn

Scikit-learn is a comprehensive open-source machine learning library for Python that provides simple and efficient tools for data mining and data analysis. In this project, Scikit-learn was used for train-test splitting (`train_test_split`), feature scaling (`StandardScaler`), model training and prediction (`LogisticRegression`), and performance evaluation (`accuracy_score`, `precision_score`, `recall_score`, `confusion_matrix`, `classification_report`).

4.2 GitHub Repository Usage

A dedicated GitHub repository was created to host all project materials for Task 5. The repository is available at <https://github.com/dbmodi2311/Data-Collection-Data-Source-Identification/tree/main/Task%205>. The repository serves three key purposes in the context of this internship.

First, version control: Git enables tracking of all changes made to the source code over time, allowing the developer to revert to previous versions, compare changes across commits, and maintain a clean audit trail of the development process. Second, project documentation: the repository includes a README.md file that provides a concise explanation of the task objectives, methodology, and results, making the project accessible to external reviewers. Third, code storage and sharing: hosting the code on GitHub makes it accessible from any device and allows submission via a stable URL that can be easily reviewed by the Edutech Solution team.

4.3 Code Explanation

The following section provides a block-by-block explanation of the Python implementation.

Block 1: Importing Libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, confusion_matrix,
                             classification_report)
```

Code Block 1: Library Imports

All required libraries are imported at the start of the script. Pandas handles data loading and preprocessing; Scikit-learn provides the train-test split, the logistic regression model, the standard scaler, and the evaluation metric functions.

Block 2: Loading and Encoding the Dataset

```
df = pd.read_csv('heart.csv')
print('Dataset Shape:', df.shape)
print(df.head())

# One-hot encode categorical columns
df = pd.get_dummies(df, drop_first=True)
```


Code Block 2: Dataset Loading and One-Hot Encoding

The dataset is loaded from the CSV file into a Pandas DataFrame. The `pd.get_dummies()` function is called on the entire DataFrame with `drop_first=True`, which automatically identifies all object-type (string) columns and replaces them with binary indicator columns. The `drop_first=True` argument drops the first category of each encoded feature to avoid multicollinearity.

Block 3: Feature and Target Selection with Train-Test Split

```
X = df.drop('HeartDisease', axis=1)
y = df['HeartDisease']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print(f'Train: {X_train.shape}, Test: {X_test.shape}')
```

Code Block 3: Feature/Target Split and Train-Test Split

The feature matrix `X` and target vector `y` are separated. The data is then split 80/20 into training and test sets with a fixed random seed for reproducibility.

Block 4: Scaling, Training, and Prediction

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

Code Block 4: Feature Scaling, Model Training, and Prediction

Features are standardised using `StandardScaler`, which is fitted on the training data only and applied to both sets, preventing data leakage. The Logistic Regression model is trained with a `max_iter` of 1000 to ensure convergence. Predictions are generated on the scaled test features.

Block 5: Evaluation Metrics

```
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print(f'Accuracy : {acc:.4f}')
```

```
print(f'Precision: {prec:.4f}')
```

```
print(f'Recall    : {rec:.4f}')
```

```
print('Confusion Matrix:')
```

```
print(cm)
```

```
print(classification_report(y_test, y_pred))
```

Code Block 5: Evaluation Metrics Computation

All standard classification metrics are computed and printed. The `classification_report` provides a multi-metric summary per class, including precision, recall, F1-score, and support.

4.4 Execution Workflow

The complete execution workflow is as follows. The dataset is first loaded from the CSV file and its shape and contents are verified. Categorical columns are one-hot encoded using `pd.get_dummies()`. The feature matrix and target vector are separated. The data is split into 80% training and 20% testing subsets with `random_state=42`. Feature values are standardised using `StandardScaler` fitted on the training set. A Logistic Regression model is instantiated and trained on the standardised training data. The trained model generates predictions on the standardised test features. Accuracy, precision, recall, and the confusion matrix are computed by comparing predicted labels against true test labels. The results are printed to the console and interpreted to assess model performance.

Chapter 5: Results and Evaluation

5.1 Model Output Summary

The Logistic Regression model was trained and evaluated on the Heart Disease Dataset using the methodology described in the preceding chapters. The following results were obtained from the model's predictions on the 184-sample test set.

5.2 Accuracy Score

Accuracy is defined as the proportion of all test predictions (both positive and negative) that were correct. It is computed as: $\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$. Substituting the obtained values: $\text{Accuracy} = (88 + 68) / (88 + 68 + 9 + 19) = 156 / 184 = 0.8478$. The model achieved an accuracy of 84.78%, indicating that approximately 85 out of every 100 test patients were classified correctly.

5.3 Precision Score

Precision measures the proportion of all patients predicted to have heart disease who were actually diagnosed with the condition. It is computed as: $\text{Precision} = TP / (TP + FP) = 88 / (88 + 9) = 88 / 97 = 0.9072$. The model achieved a precision of 90.72%, indicating that when the model predicted heart disease, it was correct approximately 91 times out of 100. This high precision reduces the likelihood of unnecessary follow-up procedures for patients who do not actually have heart disease.

5.4 Recall Score

Recall, also known as sensitivity, measures the proportion of all patients who actually have heart disease that the model successfully identified. It is computed as: $\text{Recall} = TP / (TP + FN) = 88 / (88 + 19) = 88 / 107 = 0.8224$. The model achieved a recall of 82.24%, meaning that approximately 82 out of every 100 actual heart disease cases were correctly detected. The 19 missed cases (false negatives) represent patients whose condition was not flagged — a clinically significant concern.

5.5 Confusion Matrix

The confusion matrix provides a complete breakdown of all four possible prediction outcomes for the binary classifier on the test set.

Table 2: Confusion Matrix — Logistic Regression on Test Set (n = 184)

	Predicted: No Disease (0)	Predicted: Disease (1)
Actual: No Disease (0)	True Negative (TN) = 68	False Positive (FP) = 9
Actual: Disease (1)	False Negative (FN) = 19	True Positive (TP) = 88

True Positive (TP = 88): These are patients who actually have heart disease and were correctly identified as positive by the model. These successful detections represent the core clinical value of the prediction system.

True Negative (TN = 68): These are healthy patients correctly classified as not having heart disease. They represent cases where the model avoids false alarms and the patient is spared unnecessary investigation.

False Positive (FP = 9): These are healthy patients incorrectly predicted to have heart disease. In a clinical context, these patients would be referred for further evaluation that they do not need, leading to increased healthcare costs and patient anxiety.

False Negative (FN = 19): These are patients who actually have heart disease but were incorrectly classified as healthy. False negatives are the most clinically dangerous type of error, as these patients may not receive timely treatment.

Table 3: Evaluation Metrics Summary

Metric	Formula	Value	Interpretation
Accuracy	$(TP+TN) / \text{Total}$	84.78%	Overall correct predictions across all classes
Precision	$TP / (TP + FP)$	90.72%	Reliability of positive predictions
Recall (Sensitivity)	$TP / (TP + FN)$	82.24%	Coverage of actual positive cases
F1-Score (Pos. Class)	$2 \cdot (P \cdot R) / (P + R)$	86.27%	Harmonic mean of precision and recall
True Positives	—	88	Correct heart disease detections
True Negatives	—	68	Correct healthy classifications
False Positives	—	9	Healthy patients incorrectly flagged
False Negatives	—	19	Disease cases missed by model

5.6 Model Performance Interpretation

The logistic regression model demonstrates strong overall performance on the Heart Disease Dataset. An accuracy of 84.78% and a precision of 90.72% indicate that the model is reliable in its positive predictions — it seldom raises false alarms. The recall of 82.24% suggests that the model detects the majority of true heart disease cases, though 19 patients in the test set were missed. The F1-score of 86.27% for the positive class reflects a well-balanced model that performs adequately on both precision and recall simultaneously. These results confirm that logistic regression is an effective baseline classifier for this clinical task and provides a strong foundation for further model optimisation.

Chapter 6: Discussion

6.1 Importance of Train-Test Split

The train-test split is a foundational technique in machine learning that is essential for obtaining an honest estimate of model generalisation. Without separating training and test data, a model's reported performance may be inflated by memorisation of the training set rather than the learning of true underlying patterns. The 80:20 split used in this task ensures that the model is trained on a sufficiently large portion of the data while still maintaining a representative, independently evaluated test set. This methodology is a prerequisite for any trustworthy machine learning experiment.

6.2 Importance of Accuracy

Accuracy provides a straightforward and globally interpretable measure of model performance. In this task, with a reasonably balanced class distribution (55% positive vs 45% negative), accuracy is a meaningful metric. However, accuracy alone is insufficient for clinical applications, as it does not distinguish between the costs of different error types. A model that simply predicted the majority class for every record would achieve approximately 55% accuracy without any actual learning, illustrating why accuracy must always be complemented by other metrics.

6.3 Importance of Precision

Precision is particularly important in contexts where false positives carry a significant cost. In heart disease screening, a false positive results in a healthy patient being referred for further cardiac investigation, which may be stressful, time-consuming, and expensive. A high precision of 90.72% means that the model is conservative in its positive predictions, only flagging patients as high-risk when the evidence strongly supports it. This property is valuable in resource-constrained healthcare environments.

6.4 Importance of Recall

Recall is critical in medical diagnostics where missing a true positive (a false negative) can have life-threatening consequences. A recall of 82.24% is acceptable for a baseline logistic regression model, but the 17.76% of missed heart disease cases represents a non-trivial clinical risk. In a clinical deployment scenario, the decision threshold may be lowered from 0.5 to

increase recall at the cost of precision, thereby reducing the number of missed diagnoses while accepting a higher false alarm rate.

6.5 Importance of Confusion Matrix

The confusion matrix provides a more complete and nuanced picture of model performance than any single scalar metric. By decomposing predictions into four categories — TP, TN, FP, and FN — it allows clinicians and data scientists to assess the specific types of errors the model is making. This enables informed decision-making about threshold adjustments, class weighting strategies, and model selection based on the error types that are most important to minimise in a given application.

6.6 Advantages of Logistic Regression

Logistic Regression offers numerous practical advantages that make it an appropriate first-choice model for this task. It trains quickly on datasets of this size. Its coefficients are interpretable and directly related to clinical risk factors. It produces calibrated probability outputs that can be used for risk stratification rather than just binary classification. It is supported by well-established statistical theory and is widely accepted in clinical research and regulatory contexts. It provides a reliable performance baseline against which the benefits of more complex models can be assessed.

6.7 Challenges Faced During Implementation

Several practical challenges were encountered during the implementation of this task. The presence of categorical features required careful selection of an encoding strategy. One-hot encoding with `get_dummies()` was chosen over label encoding to avoid introducing false ordinal relationships in nominal variables. Ensuring that the `StandardScaler` was fitted only on the training data — and not on the test data — required careful attention to the order of operations in the pipeline to prevent data leakage, which would invalidate the test performance metrics.

6.8 Lessons Learned

This task provided several important practical and conceptual insights. It demonstrated the end-to-end machine learning workflow from raw data to evaluated model. It reinforced the importance of the principle of data separation in model evaluation. It highlighted the complementary nature of different evaluation metrics and the need to select metrics appropriate

to the problem domain. It provided hands-on experience with industry-standard tools (Pandas, Scikit-learn, Google Colab) that are directly applicable to professional machine learning roles.

6.9 Future Improvements

Several directions for future improvement can be identified. Cross-validation (k-fold) could replace the single train-test split to obtain more robust and stable performance estimates. More powerful classifiers — such as Random Forest, Gradient Boosting, or Support Vector Machines — could be compared against the logistic regression baseline. Hyperparameter optimisation using GridSearchCV or RandomizedSearchCV could improve model performance. ROC curve analysis and the Area Under the Curve (AUC) metric would provide a threshold-independent view of model discrimination ability. Feature importance analysis could identify which clinical variables are most predictive of heart disease, providing actionable insights for clinical decision-making.

Chapter 7: Interview Questions and Answers

Question 1: What is Overfitting?

Overfitting is a phenomenon in machine learning where a model learns the training data too precisely — capturing not only the true underlying patterns but also the random noise and sampling-specific artefacts present in the training set. An overfit model effectively memorises the training data rather than generalising from it. The practical consequence is that the model performs excellently on training data (potentially achieving near-perfect accuracy) but poorly on new, unseen test data, because the patterns it has memorised do not generalise.

Overfitting typically arises when the model is too complex relative to the volume of training data, when training is allowed to continue for too many iterations, or when insufficient regularisation is applied. In the context of logistic regression, overfitting is mitigated by L2 regularisation (ridge), which penalises large coefficient values and encourages the model to distribute predictive weight more evenly across features. Other mitigation strategies include collecting more training data, applying cross-validation, and simplifying the model architecture.

Question 2: Accuracy vs Precision?

Accuracy and precision are both classification metrics derived from the confusion matrix, but they measure different aspects of model performance. Accuracy ($= (TP + TN) / \text{Total}$) measures the overall proportion of correct predictions across all classes and is most meaningful when the dataset is balanced and the costs of different error types are similar. Precision ($= TP / (TP + FP)$) measures the exactness of the model's positive predictions — specifically, among all instances the model predicted as positive, what fraction were actually positive.

The two metrics can diverge significantly in imbalanced datasets. A model that predicts the majority class for every sample might achieve high accuracy while having zero precision for the minority class. Precision is preferred in scenarios where false positives are expensive or harmful, such as in fraud detection, where blocking a legitimate transaction is costly. In this task, the model achieved accuracy of 84.78% and precision of 90.72%, indicating that its positive predictions are highly reliable even though its overall performance has room for improvement.

Question 3: What is Recall?

Recall (also called sensitivity or the true positive rate) is the proportion of actual positive instances that the model correctly identified. It is computed as $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$. Where precision asks 'of all the positives I predicted, how many were correct?', recall asks 'of all the actual positives, how many did I find?'. Recall is particularly important in medical diagnosis and other safety-critical applications where failing to identify a true positive (a false negative) carries severe consequences.

In this project, the model achieved a recall of 82.24%, meaning it detected 88 of the 107 actual heart disease cases in the test set. The 19 missed cases highlight the limitation of using a fixed 0.5 decision threshold. Lowering the threshold would increase recall (at the cost of reduced precision) by classifying more borderline cases as positive, reducing the risk of missed diagnoses.

Question 4: What is a Confusion Matrix?

A confusion matrix is a performance visualisation tool for classification models that tabulates the counts of all four possible prediction outcomes for a binary classifier. The rows of the matrix represent the actual class labels, and the columns represent the predicted class labels. The four cells contain the counts of true positives (correct positive predictions), true negatives (correct negative predictions), false positives (incorrect positive predictions, also called Type I errors), and false negatives (incorrect negative predictions, also called Type II errors).

In this project, the confusion matrix showed TP=88, TN=68, FP=9, FN=19 on a 184-sample test set. All standard classification metrics — accuracy, precision, recall, specificity, and F1-score — are derivable from these four values. The confusion matrix is indispensable for understanding the specific failure modes of a model and for making informed decisions about threshold adjustment and class weighting.

Question 5: Why Split Data?

Data splitting into training and test (and optionally validation) sets is the foundational technique that makes honest machine learning model evaluation possible. If a model is trained and evaluated on the same data, it can achieve perfect or near-perfect performance simply by memorising the training examples, without having learned any generalisable patterns. This phenomenon, known as data snooping or training-set overfitting, produces evaluation metrics that are misleading and cannot be trusted to predict real-world performance.

By withholding a portion of the data from training and using it solely for evaluation, the test set provides an unbiased simulation of real-world deployment conditions. The model has never seen these samples during training and must generalise from the patterns it learned to make accurate predictions. This ensures that reported evaluation metrics are honest indicators of the model's likely performance on new patient data encountered after deployment.

Chapter 8: Conclusion

8.1 Summary of Work

This report has documented the complete implementation of Task 5 of the Edutech Solution AI and Machine Learning Internship. The task involved loading and exploring the Heart Disease Dataset, encoding categorical features using `pd.get_dummies()`, splitting the data into 80% training and 20% testing subsets, training a Logistic Regression classifier, generating predictions on the test set, and computing a comprehensive suite of evaluation metrics including accuracy, precision, recall, and the confusion matrix.

8.2 Key Findings

The Logistic Regression model achieved an accuracy of 84.78%, a precision of 90.72%, and a recall of 82.24% on the 184-sample test set. The confusion matrix revealed 88 true positives, 68 true negatives, 9 false positives, and 19 false negatives, yielding an F1-score of 86.27% for the positive class. These results demonstrate that logistic regression is an effective and interpretable baseline classifier for heart disease prediction and that the evaluation framework employed provides a meaningful, multi-faceted assessment of model performance.

8.3 Skills Learned

Through the completion of this task, the following technical skills were developed and strengthened: data loading and inspection with Pandas; categorical feature encoding using one-hot encoding; supervised learning model training with Scikit-learn; feature scaling and the avoidance of data leakage; binary classification model evaluation using accuracy, precision, recall, F1-score, and confusion matrix; professional code documentation and version control using GitHub; and cloud-based Python development in Google Colab.

8.4 Importance of Model Evaluation

The skills acquired in this task form the bedrock of responsible machine learning practice. A model that cannot be rigorously evaluated cannot be trusted in deployment, regardless of its training performance. The ability to correctly interpret evaluation metrics, understand the clinical significance of different error types, and communicate model performance to both technical and non-technical stakeholders is an essential competency for any machine learning practitioner working in high-stakes domains such as healthcare.

8.5 Final Conclusion

Task 5 has provided a comprehensive, hands-on introduction to the train-test split methodology and classification evaluation metrics in the context of a real-world medical dataset. The logistic regression model trained on the Heart Disease Dataset achieves solid baseline performance and demonstrates the practical utility of simple, interpretable machine learning algorithms in clinical decision support. The evaluation framework employed — encompassing accuracy, precision, recall, and the confusion matrix — provides a complete and nuanced picture of model behaviour that goes well beyond simplistic accuracy reporting. These fundamentals will serve as a strong foundation for more advanced tasks in the remainder of the internship programme.

References

Dataset Kaggle: <https://www.kaggle.com/code/tanmay111999/heart-failure-prediction-cv-score-90-5-models/input>

GitHub Repository: <https://github.com/dbmodi2311/Data-Collection-Data-Source-Identification/tree/main/Task%205>

Google Colab Notebook:

https://colab.research.google.com/drive/1Dzrh7EupG_FMEAyvNVKhzGMj7YNIBifb

Appendix A: Complete Python Source Code

The following listing presents the complete Python source code used to implement Task 5.

```
# =====
# Task 5: Train-Test Split & Evaluation Metrics
# Edutech Solution AI & ML Internship
# Dataset: Heart Disease Dataset (heart.csv)
# =====

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, confusion_matrix,
                             classification_report)

# ---- Step 1: Load Dataset ----
df = pd.read_csv('heart.csv')
print('Dataset Shape:', df.shape)
print('First 5 rows:')
print(df.head())
print('\nData Types and Null Counts:')
print(df.info())

# ---- Step 2: One-Hot Encode Categorical Features ----
df = pd.get_dummies(df, drop_first=True)
print('\nShape after encoding:', df.shape)

# ---- Step 3: Separate Features and Target ----
X = df.drop('HeartDisease', axis=1)
y = df['HeartDisease']

# ---- Step 4: Train-Test Split (80:20) ----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print(f'\nTraining set size : {X_train.shape}')
print(f'Testing set size : {X_test.shape}')

# ---- Step 5: Feature Scaling ----
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train) # fit on train only
X_test = scaler.transform(X_test) # apply same scale to test

# ---- Step 6: Train Logistic Regression ----
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train, y_train)
print('\nModel training complete.')

# ---- Step 7: Predict on Test Data ----
y_pred = model.predict(X_test)

# ---- Step 8: Evaluate Model ----
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
```

```

print('\n===== Model Evaluation =====',
print(f'Accuracy : {acc:.4f} ({acc*100:.2f}%)')
print(f'Precision : {prec:.4f} ({prec*100:.2f}%)')
print(f'Recall : {rec:.4f} ({rec*100:.2f}%)')
print('\nConfusion Matrix:')
print(cm)
print('\nDetailed Classification Report:')
print(classification_report(y_test, y_pred,
target_names=['No Disease', 'Heart Disease']))

```

Appendix A: Complete Python Source Code for Task 5

Appendix B: Dataset Feature Description Table

The following table provides the complete feature description for the Heart Disease Dataset, supplementing the information provided in Chapter 2.

Table B1: Comprehensive Feature Reference Table

#	Column Name	Data Type	Encoding	Range / Categories	Medical Significance
1	Age	Numerical	None	28 – 77 yrs	Older age is a major risk factor for CVD
2	Sex	Categorical	One-hot (M/F)	M, F	Males have higher cardiovascular risk
3	ChestPainType	Categorical	One-hot	ATA, NAP, ASY, TA	ASY type strongly associated with disease
4	RestingBP	Numerical	None	0 – 200 mm Hg	Elevated BP indicates hypertension risk
5	Cholesterol	Numerical	None	0 – 603 mg/dl	High cholesterol linked to artery blockage
6	FastingBS	Binary	None	0 or 1	Elevated fasting glucose indicates diabetes
7	RestingECG	Categorical	One-hot	Normal, ST, LVH	ST/LVH anomalies suggest cardiac stress
8	MaxHR	Numerical	None	60 – 202 bpm	Lower max HR indicates reduced cardiac fitness
9	ExerciseAngina	Categorical	One-hot	Y, N	Exercise-induced pain is a disease indicator
10	Oldpeak	Numerical	None	-2.6 – 6.2	Higher values indicate ischemic changes
11	ST_Slope	Categorical	One-hot	Up, Flat, Down	Flat/Down slope associated with disease
12	HeartDisease	Binary (Target)	—	0 (Healthy), 1 (Disease)	Outcome variable for classification